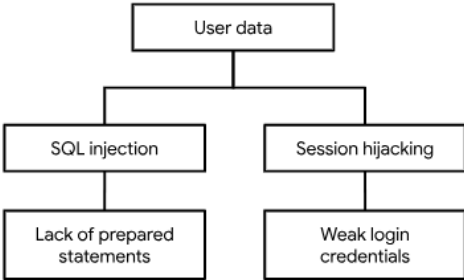


Hoja de trabajo PASTA

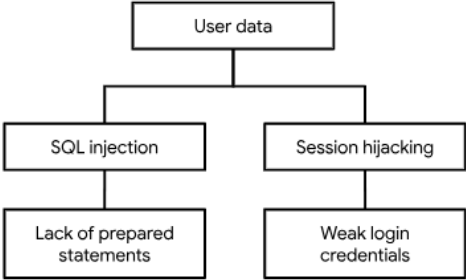
Etapas	Empresa de zapatillas
I. Definir los objetivos comerciales y de seguridad	• La aplicación procesa transacciones financieras y maneja múltiples pasarelas de pago para optimizar la experiencia del cliente. • El back-end requiere una alta capacidad de procesamiento para gestionar cuentas, privacidad de datos, historial de compras y calificación de vendedores. • Cumplimiento PCI DSS: Obligatorio para el procesamiento, transmisión y almacenamiento seguro de los datos de tarjetas de crédito/débito de los usuarios. • Cumplimiento Ley 25.326 (Protección de Datos Personales, Argentina): Exige la inscripción de las bases de datos ante la AAIP y garantizar los derechos de los usuarios a corregir, solicitar o eliminar su información personal (nombres, DNI, envíos).
II. Definir el alcance técnico	Lista de tecnologías utilizadas por la aplicación: • <i>Interfaz de programación de aplicaciones (API)</i> • <i>Infraestructura de clave pública (PKI)</i> • <i>SHA-256</i> • <i>SQL</i> Interfaz de programación de aplicaciones (API): Centraliza la lógica de negocio y el acceso a los datos, pero expande significativamente la superficie de ataque. Un token comprometido, endpoints no documentados o una mala autenticación podrían exponer datos sensibles de la base de datos. Infraestructura de clave pública (PKI): Esencial para cifrar las comunicaciones, pero una gestión deficiente, como la falta de rotación de certificados o el compromiso de una clave privada, abriría la puerta a ataques de intermediario (Man-in-the-Middle). SHA-256: Aunque es el estándar criptográfico actual y resistente a colisiones, la seguridad del sistema podría verse comprometida por errores de implementación, como su uso para almacenar contraseñas sin la adición de un "salt", haciéndolo vulnerable a ataques de diccionario. SQL: Su rigurosidad (ACID) protege la integridad de las

	transacciones, pero la base de datos es el objetivo principal de los atacantes. Representa un riesgo crítico si se introducen vectores de inyección de código o si las cuentas de servicio operan con permisos excesivos.
III. Descomponer la aplicación	Data flow diagram Note: This data flow diagram represents a single process. Data flow diagrams for an application like this are normally much more complex.  <pre> graph LR User[User] -- "Searching for sneakers for sale." --> Process((Product search process)) Process -- "Listings of current inventory." --> Database[Database] </pre>
IV. Análisis de amenazas	• Amenaza interna: Un empleado descontento o un usuario interno con credenciales comprometidas que intente extraer, alterar o destruir registros de clientes y transacciones aprovechando fallos en la asignación de permisos. • Amenaza externa: Cibercriminales que busquen comprometer el sistema mediante ataques de inyección SQL (SQLi), robo de cuentas a través del Secuestro de Sesión (Session Hijacking), o campañas de suplantación de identidad (Phishing) mediante la creación de sitios web gemelos.
V. Análisis de vulnerabilidades	• Validación insuficiente o nula de los parámetros de entrada del usuario en los formularios web y manejo inadecuado de errores visibles para el cliente. • Mala configuración en la gestión de sesiones: cookies sin atributos de seguridad, falta de regeneración de IDs tras el inicio de sesión y tiempos de caducidad inexistentes. • Exceso de privilegios en las cuentas de servicio de la base de datos y falta de segregación de funciones en los entornos de producción. • Arquitectura de red sin configuraciones de Alta Disponibilidad (HA), lo que representa un punto único de fallo ante ataques de denegación de servicio.

VI. Modelado de ataques	Sample attack tree Note: Applications like this normally have large, complex attack trees with many branches.  <pre> graph TD A[User data] --> B[SQL injection] A --> C[Session hijacking] B --> D[Lack of prepared statements] C --> E[Weak login credentials] </pre>
VII. Análisis de riesgos e impacto	• Consultas preparadas (Prepared Statements): Implementación obligatoria en el código para separar los datos del usuario de la lógica de la base de datos, neutralizando la inyección SQL. • Principio de Mínimo Privilegio (PoLP) y División de Responsabilidades: Restringir el acceso a los datos y sistemas estrictamente a lo necesario para cada rol o función específica. • Almacenamiento seguro de credenciales (Hash + SALT): Guardar las contraseñas utilizando SHA-256 junto con un valor "salt" aleatorio y único por usuario, inutilizando los datos en caso de una filtración de la base de datos. • Atributos de seguridad en Cookies (HttpOnly y Secure): Configurar HttpOnly para impedir que scripts del lado del cliente (XSS) accedan al token de sesión, y Secure para obligar al navegador a transmitir la cookie exclusivamente a través de túneles cifrados (HTTPS).

PASTA Worksheet

Stages	Sneaker Company
I. Define Business and Security Objectives	• The application processes financial transactions and handles multiple payment gateways to optimize the customer experience. • The back-end requires high processing capacity to manage accounts, data privacy, purchase history, and seller ratings. • PCI DSS Compliance: Mandatory for the secure processing, transmission, and storage of user credit/debit card data. • Compliance with Law 25.326 (Personal Data Protection, Argentina): Requires registration of databases with the AAIP and guarantees users' rights to correct, request, or delete their personal information (names, ID numbers, shipments).
II. Define Technical Scope	List of technologies used by the application: • <i>Application Programming Interface (API)</i> • <i>Public Key Infrastructure (PKI)</i> • <i>SHA-256</i> • <i>SQL</i> Application Programming Interface (API): Centralizes business logic and data access, but significantly expands the attack surface. A compromised token, undocumented endpoints, or poor authentication could expose sensitive database information. Public Key Infrastructure (PKI): Essential for encrypting communications, but poor management, such as a lack of certificate rotation or a compromised private key, would open the door to Man-in-the-Middle attacks. SHA-256: Although it is the current cryptographic standard and collision-resistant, system security could be compromised by implementation errors, such as using it to store passwords without adding a salt, making it vulnerable to dictionary attacks. SQL: Its strictness (ACID) protects transactional integrity, but the database is the primary target for attackers. It represents a critical risk if code injection vectors are introduced or if service accounts operate with excessive permissions.

III. Application Decomposition	Data flow diagram Note: This data flow diagram represents a single process. Data flow diagrams for an application like this are normally much more complex.  <pre> graph LR User[User] -- "Searching for sneakers for sale." --> Process((Product search process)) Process -- "Listings of current inventory." --> Database[Database] </pre>
IV. Threat Analysis	• Insider threat: A disgruntled employee or an internal user with compromised credentials attempting to extract, alter, or destroy customer and transaction records by exploiting flaws in permission assignments. • External threat: Cybercriminals seeking to compromise the system through SQL injection (SQLi) attacks, account takeover via Session Hijacking, or Phishing campaigns using lookalike websites.
V. Vulnerability Analysis	• Insufficient or missing input validation for user parameters in web forms, and improper handling of client-visible errors. • Misconfiguration in session management: cookies without security attributes, failure to regenerate session IDs after login, and missing expiration timeouts. • Excessive privileges in database service accounts and lack of separation of duties in production environments. • Network architecture without High Availability (HA) configurations, representing a single point of failure against denial-of-service attacks.
VI. Attack Modeling	Sample attack tree Note: Applications like this normally have large, complex attack trees with many branches.  <pre> graph TD UserData[User data] --> SQLInjection[SQL injection] UserData --> SessionHijacking[Session hijacking] SQLInjection --> LackOfPrepared[Lack of prepared statements] SessionHijacking --> WeakLogin[Weak login credentials] </pre>

VII. Risk and Impact Analysis	● Prepared Statements: Mandatory implementation in code to separate user data from database logic, neutralizing SQL injection. ● Principle of Least Privilege (PoLP) and Separation of Duties: Restrict access to systems and data strictly to what is necessary for each specific role or function. ● Secure Credential Storage (Hash + Salt): Store passwords using SHA-256 alongside a unique, random salt value per user, rendering data useless in the event of a database leak. ● Cookie Security Attributes (HttpOnly and Secure): Configure HttpOnly to prevent client-side scripts (XSS) from accessing session tokens, and Secure to force the browser to transmit cookies exclusively through encrypted channels (HTTPS).
--------------------------------------	--
